

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 7 : Les Sorties Visuelles (Visual Output)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

7.0 Introduction : Communiquer avec l'utilisateur

De l'électronique à l'Information

Un microcontrôleur qui calcule dans le vide n'est pas très utile. Il doit pouvoir informer l'utilisateur de son état, de la valeur de ses capteurs ou des erreurs rencontrées.

Dans ce chapitre, nous allons apprendre à :

- Piloter et moduler la luminosité des LEDs (PWM).
- Utiliser des transistors pour les LEDs de puissance.
- Comprendre et câbler les afficheurs à 7 segments.
- Maîtriser le principe de la Persistance Rétinienne (Multiplexage).
- Interfacer des écrans LCD alphanumériques standard et I2C.
- Créer nos propres caractères personnalisés.

7.1 La LED : Rappels fondamentaux

La Diode Électroluminescente (LED) est le composant de base de l'affichage.

Règles d'or :

1. **Polarité** : Le courant ne passe que dans un sens (de l'Anode vers la Cathode). La patte la plus longue est le PLUS (+).
2. **Protection (Loi d'Ohm)** : Une LED n'a quasiment pas de résistance interne. Branchée directement sur le 5V, elle grille instantanément ! Il FAUT une résistance en série.

Calcul de la Résistance ($R = \frac{U}{I}$)

Pour une LED rouge classique (chute de tension de 2V, courant idéal de 20mA) sur une broche Arduino (5V) :

$$R = \frac{5V - 2V}{0.02A} = \frac{3V}{0.02A} = 150\Omega.$$

En pratique, on utilise souvent une résistance de 220 Ω ou 330 Ω .

7.2 Moduler la luminosité : Le PWM

Les broches numériques de l'Arduino ne peuvent fournir que 0V ou 5V. Pour faire varier la luminosité, on utilise la **Modulation de Largeur d'Impulsion (PWM)**.

Rappel : Seules les broches marquées d'un tilde (~) supportent le PWM (broches 3, 5, 6, 9, 10, 11).

```
const int ledPin = 9; // Broche PWM

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Allumage progressif de la LED (Fade in)
  for (int luminosite = 0; luminosite <= 255; luminosite++) {
    analogWrite(ledPin, luminosite);
    delay(10); // Pause pour que l'oeil perçoive le changement
  }
}
```

7.3 Piloter des LEDs de puissance

La limite de l'Arduino : Une broche d'Arduino ne peut fournir que **40 mA maximum**. Si vous voulez piloter un bandeau LED entier ou une LED haute puissance (1 Watt), vous détruirez la broche !

La solution : Le Transistor (Bipolaire ou MOSFET) Le transistor agit comme un interrupteur électronique robuste :

- Le circuit de puissance (Alimentation 12V + Bandeau LED) traverse le transistor.
- L'Arduino n'envoie qu'un tout petit signal (5V, 1mA) sur la broche de commande (Base ou Grille) du transistor pour le fermer ou l'ouvrir.

Note : Le signal PWM de l'Arduino fonctionnera parfaitement à travers le transistor pour baisser la luminosité du bandeau 12V !

7.4 L’Afficheur 7 Segments

C'est un composant contenant simplement **8 LEDs** (7 barres + 1 point décimal) arrangées en forme de "8".

Il existe deux types d'afficheurs, qui changent la façon de les câbler et de les coder :

1. **Anode Commune (+ commun)** : Toutes les LEDs sont reliées au 5V. L'Arduino doit envoyer LOW (0V) sur une broche pour allumer un segment.
2. **Cathode Commune (- commun)** : Toutes les LEDs sont reliées au GND (Masse). L'Arduino doit envoyer HIGH (5V) pour allumer un segment.

Attention : Il faut une résistance de 220Ω pour CHAQUE segment connecté à l'Arduino (soit 7 résistances au total) !

7.5 Code : Afficher un chiffre (7 Segments)

Pour afficher le chiffre "1", on doit allumer les segments B et C.

```
// Un tableau pour simplifier la correspondance des broches (Segments A a G)
const int brochesSegments[7] = {2, 3, 4, 5, 6, 7, 8};

void setup() {
  for (int i = 0; i < 7; i++) {
    pinMode(brochesSegments[i], OUTPUT);
  }
}

void loop() {
  // Exemple pour un afficheur a Cathode Commune : Allumer le chiffre "1"
  // On allume les segments aux index 1 (B) et 2 (C)
  digitalWrite(brochesSegments[0], LOW);  // A
  digitalWrite(brochesSegments[1], HIGH); // B
  digitalWrite(brochesSegments[2], HIGH); // C
  digitalWrite(brochesSegments[3], LOW);  // D
  // ... etc
}
```

7.6 Le Multiplexage Temporel

Le Problème : Un afficheur à 1 chiffre prend 7 broches. Pour afficher une heure (4 chiffres), il faudrait $4 \times 7 = 28$ broches ! L'Arduino Uno n'en a que 14.

La Solution : La Persistance Rétinienne On connecte tous les segments A ensemble, tous les B ensemble, etc. Puis on ajoute un transistor pour activer ou désactiver chaque chiffre entier.

- Étape 1 : Activer le chiffre des milliers, envoyer le code pour le chiffre "1", attendre 5 ms.
- Étape 2 : Désactiver les milliers, activer les centaines, envoyer le code pour "9", attendre 5 ms.
- Répéter très vite. L'œil humain (trop lent) verra les 4 chiffres allumés simultanément !

7.7 Les Écrans LCD Alphanumériques

Pour afficher du texte compréhensible (ex : "Temp : 24.5C"), le 7 segments ne suffit plus. On utilise les **Afficheurs LCD basés sur le contrôleur HD44780**.

Caractéristiques Standards :

- Ils existent en différentes tailles : 16x2 (16 colonnes, 2 lignes) ou 20x4.
- Ils possèdent 16 broches (Alimentation, Rétroéclairage, Contraste, Données).
- Ils peuvent fonctionner en "Mode 4 bits" : on n'utilise que 4 broches de données (D4 à D7) au lieu de 8, pour économiser les broches de l'Arduino.

7.8 Le Câblage Standard du LCD (Mode 4 Bits)

Câbler un LCD HD44780 directement demande beaucoup de fils.

Les broches essentielles à relier à l'Arduino :

- **RS (Register Select)** : Indique si on envoie une commande (ex : "effacer l'écran") ou une donnée (ex : la lettre 'A').
- **E (Enable)** : La broche d'horloge pour valider l'envoi.
- **D4, D5, D6, D7** : Les 4 broches pour transférer le texte.

Ne pas oublier : La broche V0 (Contraste) **doit** être reliée à un potentiomètre de $10k\Omega$, sinon l'écran apparaîtra complètement blanc ou noirci !

7.9 Code : Bibliothèque LiquidCrystal

L'Arduino possède une bibliothèque native pour masquer toute la complexité de l'envoi de données vers l'écran.

```
#include <LiquidCrystal.h>

// Initialisation : LiquidCrystal(RS, E, D4, D5, D6, D7)
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  // Declaration du format de l'ecran (16 colonnes, 2 lignes)
  lcd.begin(16, 2);

  // Affiche un message a la position par defaut (0, 0)
  lcd.print("Bonjour M1 !");
}

void loop() {
  // Place le curseur sur la colonne 0, ligne 1 (la 2eme ligne)
  lcd.setCursor(0, 1);
  lcd.print(millis() / 1000); // Affiche le chronometre
  lcd.print(" s");
}
```

7.10 La Révolution de l'I2C pour les LCD

Le câblage standard du LCD prend **6 broches** et fait un "plat de spaghettis" sur la breadboard.

Le Module I2C (ex : puce PCF8574) : C'est un petit module à souder à l'arrière du LCD. Il convertit le protocole parallèle de l'écran en un protocole série **I2C**.

Avantage massif : Vous n'avez plus besoin que de **4 fils** (dont 2 pour l'alimentation) pour relier l'écran à l'Arduino :

- **GND** au GND
- **VCC** au 5V
- **SDA (Data)** à la broche A4 (Uno)
- **SCL (Clock)** à la broche A5 (Uno)

7.11 Code : Utiliser un LCD en I2C

Il faut télécharger une bibliothèque tierce, souvent nommée `LiquidCrystal_I2C.h`.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Definition de l'adresse I2C (souvent 0x27 ou 0x3F), et de la taille
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  lcd.init();           // Initialise l'ecran I2C
  lcd.backlight();      // Allume le retroeclairage

  lcd.setCursor(2, 0);  // Decale un peu le texte
  lcd.print("Systeme OK.");
}

void loop() {
  // Meme logique que l'ecran standard
}
```

7.12 Créer des Caractères Personnalisés

Le contrôleur HD44780 possède une petite RAM interne (CGRAM) qui permet de dessiner jusqu'à **8 caractères sur mesure** (pour afficher un thermomètre, un cœur, le symbole degré ° ou des accents).

Chaque caractère est une grille de 5x8 pixels. On utilise un tableau de byte.

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

// Dessin d'un petit coeur (1 = allume, 0 = eteint)
byte coeur[8] = {
  B00000,
  B01010,
  B11111,
  B11111,
  B01110,
  B00100,
  B00000,
};
```

```
void setup() {
  lcd.begin(16, 2);
```

```
// Stocke le dessin dans la case memoire 0
```

7.13 Les Matrices de LEDs

Pour afficher des animations complexes ou du texte défilant, on utilise des matrices (ex : 8x8 LEDs).

Le principe (Row-Column Scanning) : Au lieu d'avoir 64 fils, les LEDs sont reliées en grilles (8 lignes, 8 colonnes). Pour allumer la LED (2,3), on applique 5V sur la ligne 2, et le GND sur la colonne 3.

La complexité logicielle

Piloter une matrice directement avec l'Arduino nécessite de rafraîchir la grille des centaines de fois par seconde. Pour se simplifier la vie, on délègue cette tâche à une puce dédiée, comme le **MAX7219** (vu au Chapitre 18), contrôlable facilement via le bus SPI.

- **Énergie** : Ne tirez jamais trop de courant des broches numériques. Utilisez la Loi d'Ohm pour calculer vos résistances, et des Transistors pour les charges lourdes.
- **7 Segments** : Comprenez la différence entre Anode Commune (0V pour allumer) et Cathode Commune (5V pour allumer).
- **Illusion optique** : Le multiplexage temporel est la base de tous les affichages modernes. On trompe l'œil humain en allumant/éteignant très vite les segments.
- **Le Standard LCD** : La bibliothèque `LiquidCrystal` est universelle. Pensez toujours à utiliser un module **I2C** pour libérer vos broches pour les capteurs !

Fin du Chapitre 7. Le Chapitre 8 abordera la lecture de données complexes et les capteurs avancés !